MACHINE PROBLEM 1

Prepared by: Jimenez, Cyrus L. – 2022-09325-MN-0

Title: String Manipulation Toolkit

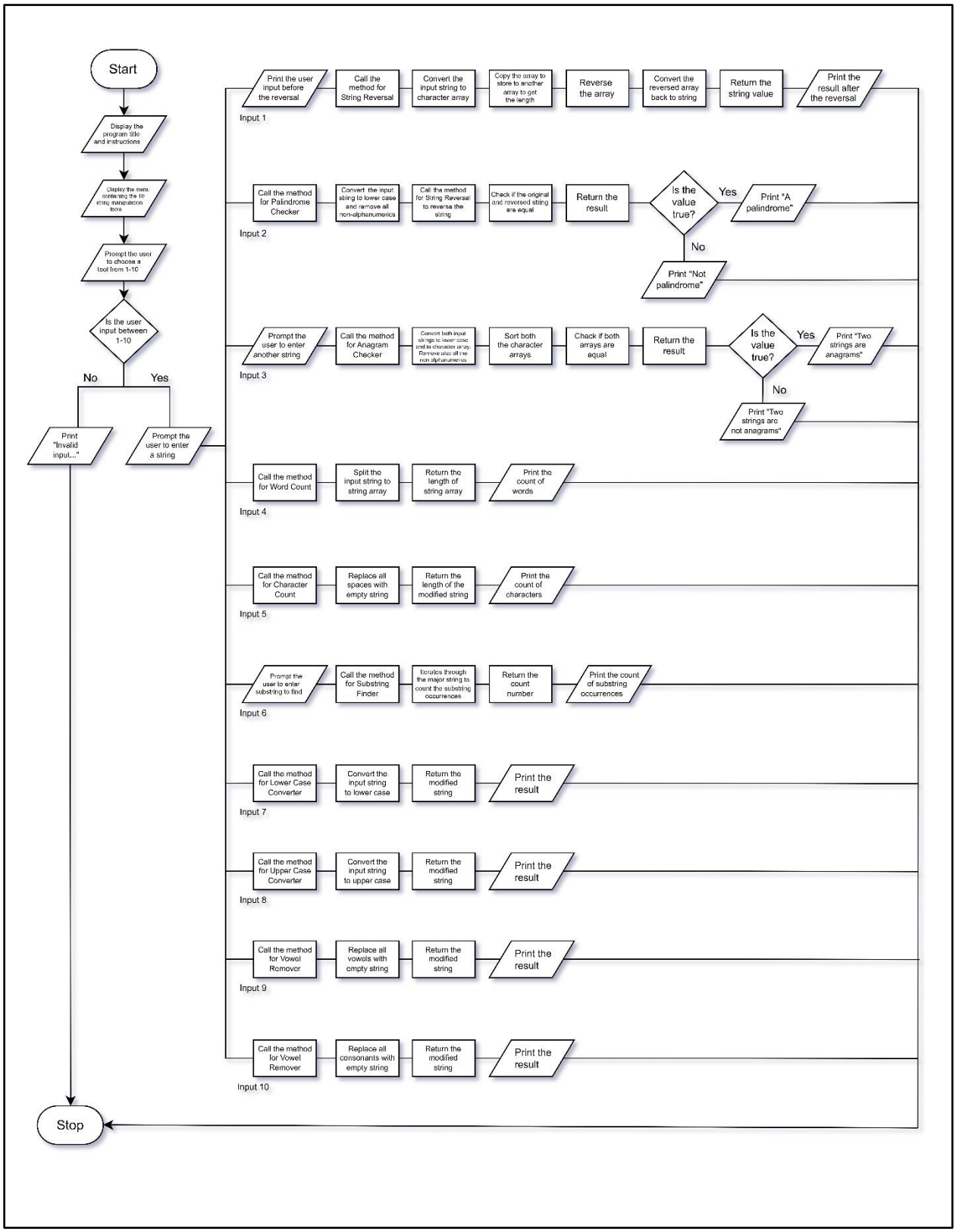
Overview:
Create a Java program that serves as a toolkit for string manipulation. The toolkit should include a set of functions that perform various operations on strings. Each method should focus on a specific aspect of string manipulation, utilizing relevant Java string methods.

Development Requirements:
You are tasked with creating a text analysis tool in Java. The tool should perform various analyses on a given text and provide relevant information. Implement the following functionalities:

1. **String Reversal:** Implement a method that reverses a given string.
2. **Palindrome Checker:** Implement a method that checks whether a given string is a palindrome.
3. **Anagram Checker:** Implement a function that checks if two strings are anagrams.
4. **Word Count:** Implement a function that counts the number of words in a given string.
5. **Character Count:** Implement a method that counts the total number of characters in the text (excluding spaces).
6. **Substring Finder:** Implement a method that finds all occurrences of a specified substring in a given string.
7. **Lower Case Converter:** Implement a method that converts a given string to lowercase.
8. **Upper Case Converter:** Implement a method that converts a given string to uppercase.
9. **Vowel Remover:** Implement a method that removes all vowels from a given string.
10. **Consonant Remover:** Implement a function that removes all consonant from a given string.

The program should allow the user to choose a Sting Manipulation Tool in the given options in the screen. The user should only input 1-10 to choose on any of the option. Any numbers aside from 1-10, should not be accepted by the program. Once the user already pick a number, the program will ask the user again to enter the string consisting of 3 sentences and this should be processed depending on the number picked by the user.

Process Flow Diagram:



Variable Definition:

- arrTools - String array containing the string manipulation tools to be displayed with the use of for loop for better readability and minimize redundancy of print statement in the source code.
- i - Integer variable type used as the initialization in the for loops.
- intMenu - Integer variable to store the chosen tool of the user based on the numbers 1-10.
- strUserInput - String variable to store the major user input string.
- blnPalindrome - Boolean variable to store the result of the palindrome check whether if it is true or false.
- strUserInput2 - String variable to store the second user input for anagram checking.
- blnAnagram - Boolean variable to store the result of the anagram check whether if it is true or false.
- strSub - String variable to store the second user input for substring finding.
- strUserString - String variable used as the major actual parameter for every user-defined method. This receives the value from the variable argument strUserInput.
- strUserInput2 - String variable used as the second actual parameter in the method for anagram checker. This receives the value from the variable argument strUserInput2 in the main class.
- strUserSub – String variable used as the second actual parameter in the method for substring finder. This receives the value from the variable argument strSub in the main class.
- arrUserInput - Character array used in the first user-defined method to store the input string converted to a character array.
- arrReversedString - Character array used in the stringReverse method to store the reversed string.
- strAlphaNumeric - String variable used in the checkPalindrome method to store the input string without non-alphanumeric characters.
- strReversedString - String variable used in the checkPalindrome method to store the reversed alphanumeric string.
- arrUserInput2 - Character array used in the checkAnagram method to store the second user input string converted to a character array.
- arrWords - String array used in the wordCounter method to store the words obtained by splitting the input string.
- strNoSpace - String variable used in the charCounter method to store the input string with no whitespaces.
- intCount - Integer variable used in the findSubOccur method to store the count of substring occurrences.
- intIndex - Integer variable used in the findSubOccur method to store the starting index for substring finding.

Method Definition:

- **stringReverse**
  String
  String strUserString
  Reverses the input string by converting it first to character array, then reverses the array using for-loop for assignation of every character, then converts it again to string, and finally returns the reversed string or value.

```
public static String stringReverse(String strUserString) {
        char[] arrUserInput = strUserString.toCharArray();
        char[] arrReversedString = new char[arrUserInput.length];

        for (int i = 0; i < arrUserInput.length; i++) {
            arrReversedString[i] = arrUserInput[arrUserInput.length - 1 - i];
        }

        return new String(arrReversedString);
    }
```

- **checkPalindrome**
  boolean
  String strUserString
  Checks whether the input string is a palindrome or not by converting it to lowercase and removing the non-alphanumeric characters to make it not case sensitive, then reversing it using the “stringReverse” method, and finally checks if the original and reversed versions are equal and returns its logic value.

```
public static boolean checkPalindrome(String strUserString) {
        String strAlphaNumeric = strUserString.replaceAll("[^a-zA-z0-9]",
"").toLowerCase();

        String strReversedString = stringReverse(strAlphaNumeric);

        return strAlphaNumeric.equals(strReversedString);
    }
```

- **checkAnagram**
  boolean
  String strUserInput, String strUserInput2
  Checks whether the two input strings are anagrams. It is done through removing first all the non-alphanumeric characters and converting the two strings to lowercase and to character array using replaceAll, toLowerCase, and toCharArray built-in methods in order. Then sorts the two converted arrays in default order using the sort built-in method. It then checks if the two sorted arrays are equal and finally returns its logic value.

```
public static boolean checkAnagram(String strUserInput, String strUserInput2) {
```

```
        char[] arrUserInput = strUserInput.replaceAll("[^a-zA-z0-9]",
"").toLowerCase().toCharArray();
        char[] arrUserInput2 = strUserInput2.replaceAll("[^a-zA-z0-9]",
"").toLowerCase().toCharArray();

        Arrays.sort(arrUserInput);
        Arrays.sort(arrUserInput2);

        return Arrays.equals(arrUserInput, arrUserInput2);
    }
```

- **wordCounter**
  int
  String strUserString
  Counts the number of words in the input string through splitting it into string array in accordance with the whitespaces it has. Finally, it then returns the length value of the string array with the use of length built-in method.

```
public static int wordCounter(String strUserString) {
        String[] arrWords = strUserString.split("\\s+");

        return arrWords.length;
    }
```

- **charCounter**
  int
  String strUserString
  Counts the number of characters in the input string excluding whitespaces by replacing all space characters with empty string. It then returns the length value or the count of character of the modified string.

```
public static int charCounter(String strUserString) {
        String strNoSpace = strUserString.replaceAll("\\s", "");

        return strNoSpace.length();
    }
```

- **findSubOccur**
  int
  String strUserString, String strUserSub
  Counts the number of substring occurrence in the input string. The steps include while loop iteration through the input string using indexOf built-in method to find occurrences of a specified substring and increments a counter for each match. It then continues until no more occurrences are found. Finally, it returns its value or the count number of the occurrences.

```
public static int findSubOccur(String strUserString, String strUserSub) {
        int intCount = 0, intIndex = -1;
```

```
        while ((intIndex = strUserString.indexOf(strUserSub, intIndex + 1)) != -1) {
            intCount++;
        }

        return intCount;
    }
```

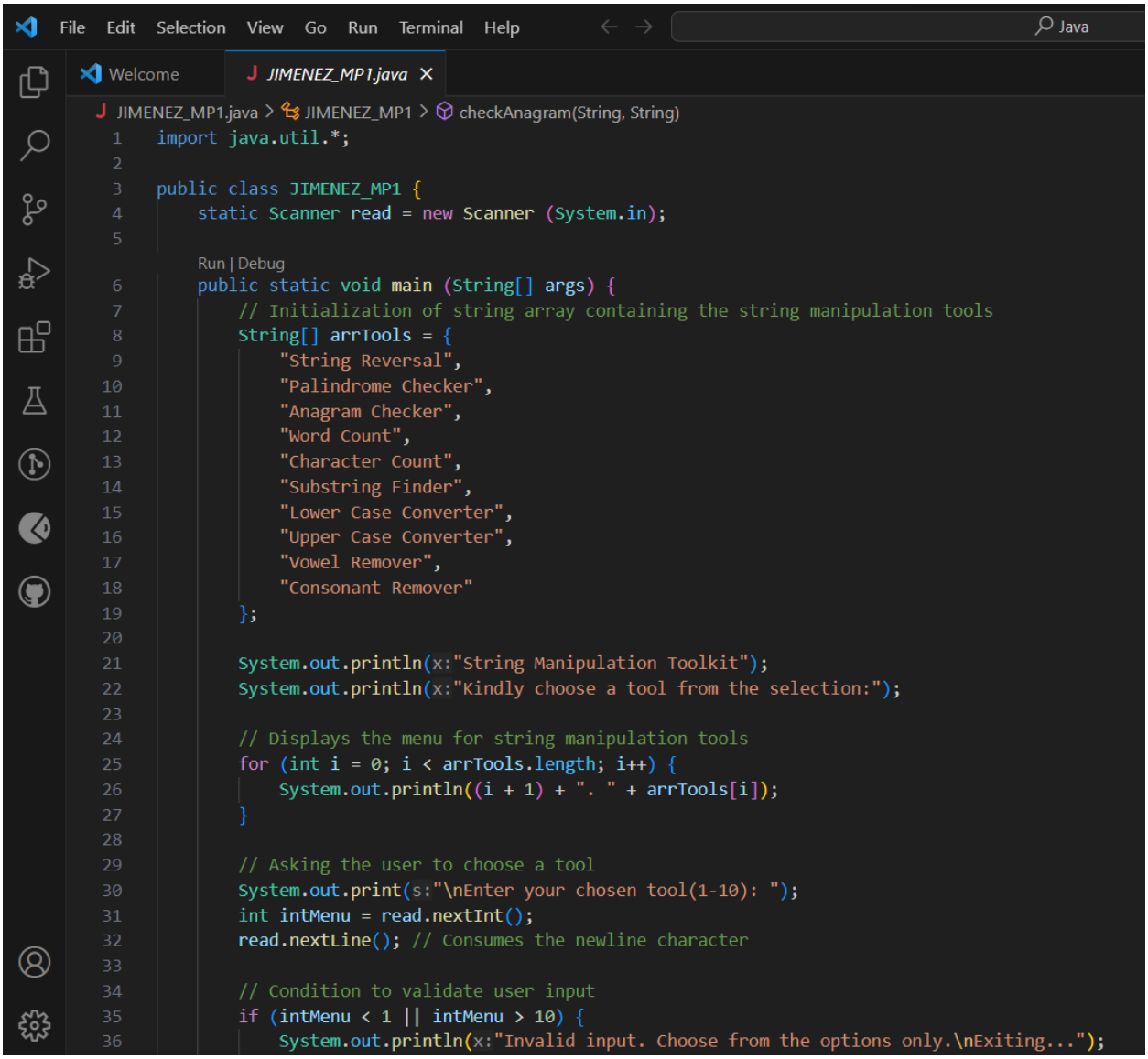
- **makeItLow**
  String
  String strUserString
  Converts the input string to all lower-case string by using the toLowerCase built-in method and returns the newly modified version of string to the main method.
  ```
  public static String makeItLow(String strUserString) {
        return strUserString.toLowerCase();
    }
  ```

- **makeItUp**
  String
  String strUserString
  Converts the input string to all upper-case string by using the toUpperCase built-in method and returns the newly modified version of string to the main method.
  ```
  public static String makeItUp(String strUserString) {
        return strUserString.toUpperCase();
    }
  ```

- **removeVowel**
  String
  String strUserString
  Removes all the vowels for both upper and lower cases in the input string by replacing the vowels with empty strings. It then returns the newly modified version of string to the main method.
  ```
  public static String removeVowel(String strUserString) {
        return strUserString.replaceAll("[aeiouAEIOU]", "");
    }
  ```

- **removeConsonant**
  String
  String strUserString
  Removes all the consonants for both upper and lower cases in the input string by replacing the consonants with empty strings. It then returns the newly modified version of string to the main method.
  ```
  public static String removeConsonant(String strUserString) {
        return
  strUserString.replaceAll("[bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ]", "");
    }
  ```

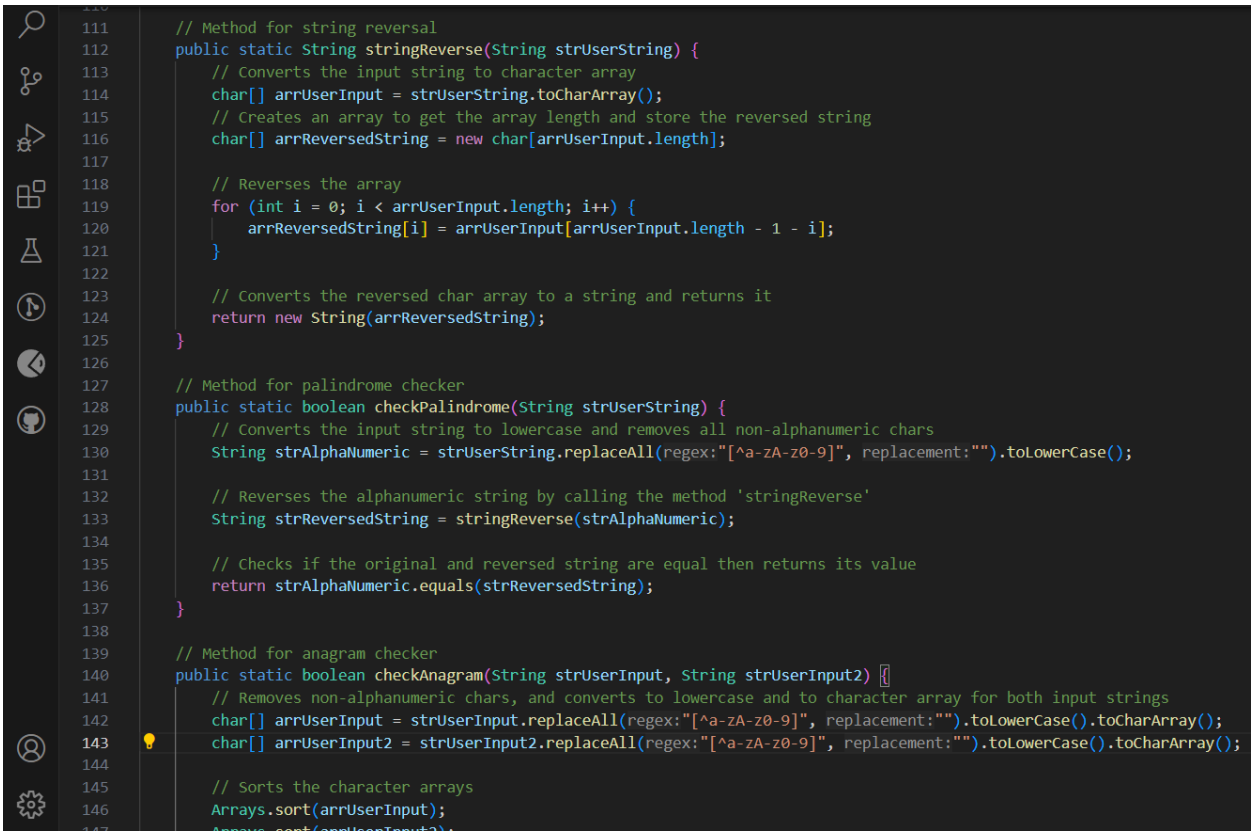
Developer Notes:



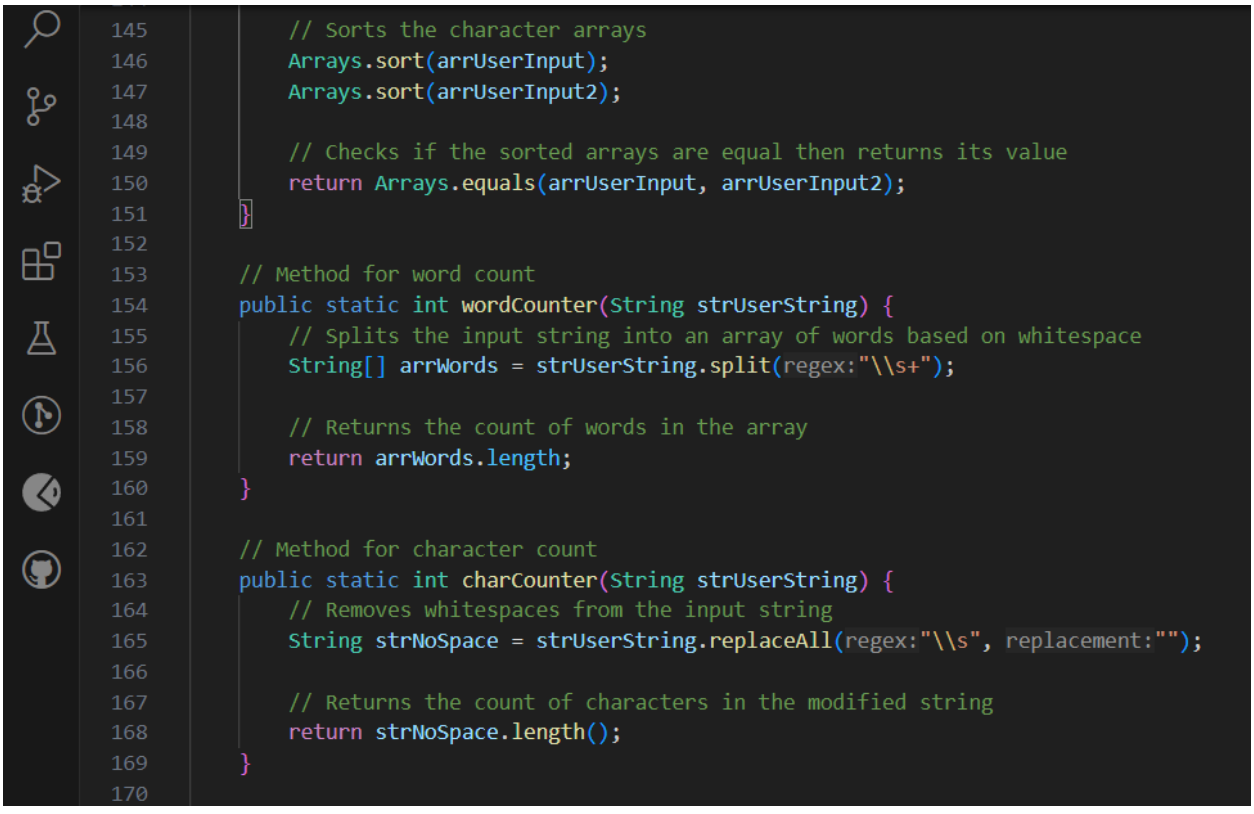

```java
import java.util.*;

public class JIMENEZ_MP1 {
    static Scanner read = new Scanner (System.in);

    public static void main (String[] args) {
        // Initialization of string array containing the string manipulation tools
        String[] arrTools = {
            "String Reversal",
            "Palindrome Checker",
            "Anagram Checker",
            "Word Count",
            "Character Count",
            "Substring Finder",
            "Lower Case Converter",
            "Upper Case Converter",
            "Vowel Remover",
            "Consonant Remover"
        };

        System.out.println(x:"String Manipulation Toolkit");
        System.out.println(x:"Kindly choose a tool from the selection:");

        // Displays the menu for string manipulation tools
        for (int i = 0; i < arrTools.length; i++) {
            System.out.println((i + 1) + ". " + arrTools[i]);
        }

        // Asking the user to choose a tool
        System.out.print(s:"\nEnter your chosen tool(1-10): ");
        int intMenu = read.nextInt();
        read.nextLine(); // Consumes the newline character

        // Condition to validate user input
        if (intMenu < 1 || intMenu > 10) {
            System.out.println(x:"Invalid input. Choose from the options only.\nExiting...");
            return;
        }

        // Asking the user to enter a string
        System.out.print(s:"Enter string with three(3) sentences: ");
        String strUserInput = read.nextLine();

        switch (intMenu) {
        case 1:
            // Calls the method 'stringReverse' directly to display result
            System.out.println("\nBefore reversing: " + strUserInput);
            System.out.println("After reversing: " + stringReverse(strUserInput));
            break;
        case 2:
            // Calls the method 'checkPalindrome' directly to store its result to variable
            boolean blnPalindrome = checkPalindrome(strUserInput);
```

```
54              // Displays palindrome status based on the boolean variable
55              if (blnPalindrome) {
56                  System.out.println(x:"A palindome");
57              } else {
58                  System.out.println(x:"Not palindrome");
59              }
60              break;
61          case 3:
62              // Asking the user to enter another string for anagram checking
63              System.out.print(s:"Enter another string: ");
64              String strUserInput2 = read.nextLine();
65
66              // Calls the method 'checkAnagram' directly to store its result to variable
67              boolean blnAnagram = checkAnagram(strUserInput, strUserInput2);
68
69              // Displays anagram status based on the boolean variable
70              if (blnAnagram) {
71                  System.out.println(x:"Two strings are anagrams");
72              } else {
```

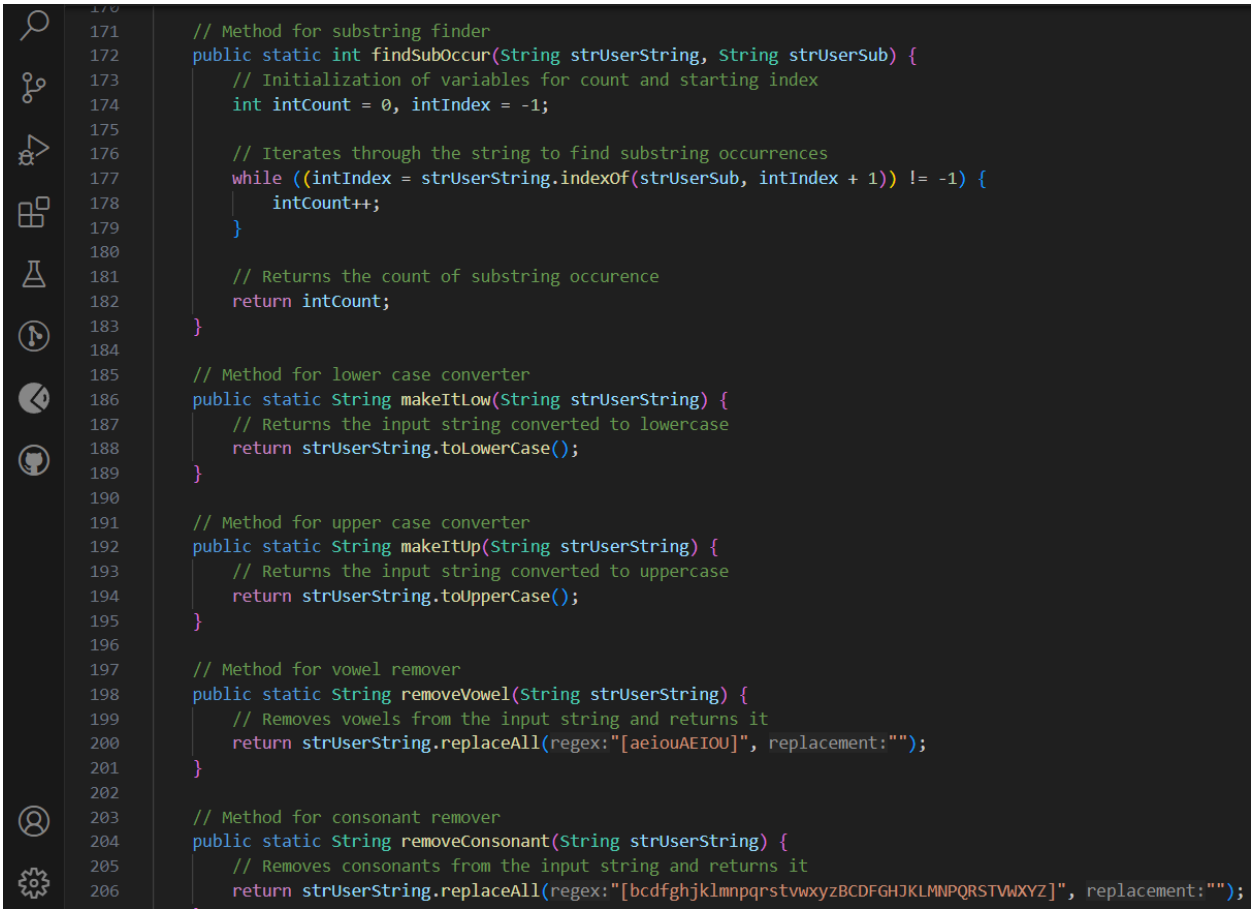
```
76          case 4:
77              // Calls the method 'wordCounter' directly to display result
78              System.out.println("Number of words: " + wordCounter(strUserInput));
79              break;
80          case 5:
81              // Calls the method 'charCounter' directly to display result
82              System.out.println("Number of characters excluding spaces: " + charCounter(strUserInput));
83              break;
84          case 6:
85              // Asking the user to enter substring
86              System.out.print(s:"Enter substring to find: ");
87              String strSub = read.nextLine();
88
89              // Calls the method 'findSubOccur' directly to display result
90              System.out.println("Number of entered substring occurence: " + findSubOccur(strUserInput, strSub));
91              break;
92          case 7:
93              // Calls the method 'makeItLow' directly to display result
94              System.out.println("After converting to lower case: " + makeItLow(strUserInput));
95              break;
96          case 8:
97              // Calls the method 'makeItUp' directly to display result
98              System.out.println("After converting to upper case: " + makeItUp(strUserInput));
99              break;
100         case 9:
101             // Calls the method 'removeVowel' directly to display result
102             System.out.println("After removing the vowels: " + removeVowel(strUserInput));
103             break;
104         case 10:
105             // Calls the method 'removeConsonant' directly to display result
106             System.out.println("After removing the consonants: " + removeConsonant(strUserInput));
107             break;
108         }
109     }
```



```java
    // Method for string reversal
    public static String stringReverse(String strUserString) {
        // Converts the input string to character array
        char[] arrUserInput = strUserString.toCharArray();
        // Creates an array to get the array length and store the reversed string
        char[] arrReversedString = new char[arrUserInput.length];

        // Reverses the array
        for (int i = 0; i < arrUserInput.length; i++) {
            arrReversedString[i] = arrUserInput[arrUserInput.length - 1 - i];
        }

        // Converts the reversed char array to a string and returns it
        return new String(arrReversedString);
    }

    // Method for palindrome checker
    public static boolean checkPalindrome(String strUserString) {
        // Converts the input string to lowercase and removes all non-alphanumeric chars
        String strAlphaNumeric = strUserString.replaceAll(regex:"[^a-zA-z0-9]", replacement:"").toLowerCase();

        // Reverses the alphanumeric string by calling the method 'stringReverse'
        String strReversedString = stringReverse(strAlphaNumeric);

        // Checks if the original and reversed string are equal then returns its value
        return strAlphaNumeric.equals(strReversedString);
    }

    // Method for anagram checker
    public static boolean checkAnagram(String strUserInput, String strUserInput2) {
        // Removes non-alphanumeric chars, and converts to lowercase and to character array for both input strings
        char[] arrUserInput = strUserInput.replaceAll(regex:"[^a-zA-z0-9]", replacement:"").toLowerCase().toCharArray();
        char[] arrUserInput2 = strUserInput2.replaceAll(regex:"[^a-zA-z0-9]", replacement:"").toLowerCase().toCharArray();

        // Sorts the character arrays
        Arrays.sort(arrUserInput);
```



```java
        // Sorts the character arrays
        Arrays.sort(arrUserInput);
        Arrays.sort(arrUserInput2);

        // Checks if the sorted arrays are equal then returns its value
        return Arrays.equals(arrUserInput, arrUserInput2);
    }

    // Method for word count
    public static int wordCounter(String strUserString) {
        // Splits the input string into an array of words based on whitespace
        String[] arrWords = strUserString.split(regex:"\\s+");

        // Returns the count of words in the array
        return arrWords.length;
    }

    // Method for character count
    public static int charCounter(String strUserString) {
        // Removes whitespaces from the input string
        String strNoSpace = strUserString.replaceAll(regex:"\\s", replacement:"");

        // Returns the count of characters in the modified string
        return strNoSpace.length();
    }
```



```
    // Method for substring finder
    public static int findSubOccur(String strUserString, String strUserSub) {
        // Initialization of variables for count and starting index
        int intCount = 0, intIndex = -1;

        // Iterates through the string to find substring occurrences
        while ((intIndex = strUserString.indexOf(strUserSub, intIndex + 1)) != -1) {
            intCount++;
        }

        // Returns the count of substring occurence
        return intCount;
    }

    // Method for lower case converter
    public static String makeItLow(String strUserString) {
        // Returns the input string converted to lowercase
        return strUserString.toLowerCase();
    }

    // Method for upper case converter
    public static String makeItUp(String strUserString) {
        // Returns the input string converted to uppercase
        return strUserString.toUpperCase();
    }

    // Method for vowel remover
    public static String removeVowel(String strUserString) {
        // Removes vowels from the input string and returns it
        return strUserString.replaceAll(regex:"[aeiouAEIOU]", replacement:"");
    }

    // Method for consonant remover
    public static String removeConsonant(String strUserString) {
        // Removes consonants from the input string and returns it
        return strUserString.replaceAll(regex:"[bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ]", replacement:"");
```

Learning Outcomes

This machine problem activity is in fact the first time that I utilize the comments feature to its maximum capacity in the integrated development environment since I do not usually write notes in my programming projects because of the thought that it is something that is tiresome and unnecessary thing to do. But throughout in this coding process, I have realized the great importance of making the use of the said attribute that it is somehow makes the source code easier to comprehend. On a more technical aspect, I have learned how to construct my own user-defined methods in this particular programming language and helps me to refresh the lessons from the past school year about functions in C language. The major dilemma that I have faced while working with the problem is the point where I had to repeat the whole coding process because I forgot to read the instructions inside the documentation and just directly based my workflow on what is posted in the Google Classroom learning management system. Fortunately, I had only finished up to the first toolkit before realizing that I made a mistake in terms of the development requirements. The frustration that I had to endure paves a way for me to take the steps slowly and patiently. These enlightenments can be implemented in most circumstances involving tasks with gradational process especially those long ones such as thesis and experiments as it contributes to the dissection of work and break down operation to emphasize each step in a project in the most convenient manner.